

Game Character Designer - Offline Visual Interface

Design an offline visual interface (desktop-like web application) for creating, editing, and managing the design data of my RTS game. The interface should be much easier to use than editing a large Excel spreadsheet directly.

The application can be built using any of the following approaches, depending on what you think is most suitable:

- HTML + CSS + JavaScript
- Python (Flask, FastAPI, or similar) with HTML
- Python desktop GUI (PySide6, PyQt, Tkinter, etc.)
- Any other offline solution that does not require an internet connection.

Data Management

The application should automatically maintain an Excel workbook that stores all game data.

- Every modification should immediately update the Excel file (auto-save), or at least keep changes in memory until the user clicks ****Save****.
- There should also be a dedicated ****Save**** button for manually saving all changes.
- The save location should be configurable.

Hero Management

The interface should allow me to:

- Create a new Hero.
- Edit an existing Hero.
- Delete a Hero.
- Rename a Hero.
- Duplicate an existing Hero.
- Assign a picture or portrait to each Hero.
- Display the Hero's image while editing.

Age Management

The game contains multiple Ages.

Examples:

- Stone Age
- Bronze Age
- Iron Age
- Medieval Age
- etc.

The interface should allow me to:

- Create new Ages.
- Rename Ages.

- Delete Ages.
- Move Heroes between Ages.
- View all Heroes belonging to a selected Age.

Editable Features

Every Hero contains a large number of editable properties.

Examples include:

- Temple
- Goddess
- Unit Type
- Attack
- Defense
- Health
- Speed
- Range
- Armor
- Cost
- Population
- Healing Ability
- Area of Effect
- Mobility
- Special Skills
- Passive Abilities
- Ultimate Ability

These are only examples.

The interface should allow me to:

- Add entirely new feature fields.
- Delete existing feature fields.
- Rename feature fields.
- Edit feature values.
- Group related features into collapsible categories.

I do not want the feature list to be hard-coded.

Image Support

Each Hero should support:

- Portrait image
- Optional additional images
- Preview inside the editor

Navigation

The interface should have:

- Previous Hero
- Next Hero

Changing Heroes should preserve all edits.

Comparison Mode

One of the most important features is Hero comparison.

The application should include a Compare Heroes page where I can:

- Select two or more Heroes.
- Display them side-by-side.
- Compare every statistic in a table.
- Highlight differences between Heroes.
- Compare images as well.

Search and Filtering

The application should include:

- Hero search
- Filter by Age
- Filter by Hero Type
- Filter by custom properties

User Interface

The interface should be visual rather than spreadsheet-like.

Suggested layout:

- Left sidebar:
 - Ages
 - Hero list
- Main panel:
 - Hero image
 - Hero information
 - Editable statistics
- Right sidebar:
 - Quick navigation
 - Feature categories

The interface should feel similar to a game editor rather than Excel.

Storage

The application should maintain:

- An Excel workbook for all game data.
- A folder containing Hero images.
- Configuration files if necessary.

The interface should automatically load the previous project when reopened.

Expected Deliverables

If building the complete application is too large for one response, generate it in phases.

Preferred folder structure:

RTS_Game_Designer

■

■ ■ ■ ■ app.py # Entry point

■ ■ ■ ■ config.py # App configuration

■ ■ ■ ■ requirements.txt

■

■ ■ ■ ■ database/

■ ■ ■ ■ ■ game.db

■ ■ ■ ■ ■ database.py

■

■ ■ ■ ■ models/

■ ■ ■ ■ ■ hero.py

■ ■ ■ ■ ■ age.py

■ ■ ■ ■ ■ property.py

■ ■ ■ ■ ■ image.py

■

■ ■ ■ ■ services/

■ ■ ■ ■ ■ hero_service.py

■ ■ ■ ■ ■ age_service.py

■ ■ ■ ■ ■ excel_service.py

■ ■ ■ ■ ■ image_service.py

■ ■ ■ ■ ■ property_service.py

■

■ ■ ■ ■ ui/

■ ■ ■ ■ ■ main_window.py

■ ■ ■ ■ ■ hero_editor.py

■ ■ ■ ■ ■ age_panel.py

■ ■ ■ ■ ■ hero_list.py

■ ■ ■ ■ ■ toolbar.py

■ ■ ■ ■ ■ compare_window.py

■ ■ ■ ■ ■ widgets/

■

■ ■ ■ ■ assets/

■ ■ ■ ■ ■ hero_images/

■ ■ ■ ■ ■ icons/

■

■ ■ ■ ■ exports/

■

■ ■ ■ ■ styles/

■ ■ ■ ■ ■ style.qss

Build the application in a modular, maintainable way with clean code and clear separation of responsibilities. The interface should be scalable enough to handle hundreds of Heroes and dozens of editable properties without becoming difficult to manage.